

串

1.串的定义和基本操作

定义

串是零个或多个字符组成的有限序列

$s = "a_1 a_2 a_3 \dots a_n"$ ($n > 0$)

s: 串的名

"a₁a₂a₃...a_n": 串的值

a_i: 字符 (字母、数字.....)

n: 串的长度

基本操作

StrAssign (&T,chars) : 赋值操作。将串T赋值为chars。
StrCopy (&T,s) : 复制操作。将串s复制给串T。
StrEmpty (s) : 判空操作。若s为空串, 则返回TRUE, 否则返回FALSE。
strcmpare (s,t) : 比较操作。若s>t, 返回值>0; 若s=t, 返回值=0; 若s<t, 返回值<0。
StrLength (s) : 求串长操作。返回串s中字符的个数。
substring (&Sub,s,pos,len) : 求子串操作。用Sub返回串s中从第pos个字符起、长度为len的子串。
Concat (&T,s1,s2) : 串联接操作。用T返回由s1 s2联接而成的新串。
Index (s,t) : 定位操作。若主串s中存在与t值相同的子串, 则返回其在s中首次出现的位置; 否则返回0。
ClearString (&s) : 清空操作。将串s置为空串。
Destroystring (&s) : 销毁操作。将串s销毁。

2.串的存储结构

顺序存储

顺序存储

```
// 串的顺序存储
#define MAXLEN 255 // 串的最大长度
typedef struct{
    char ch[MAXLEN + 1]; // 存储串的一维数组
    int length; // 串的当前长度
}SString;
```

堆式顺序存储

```
// 串的堆式顺序存储
typedef struct{
    char *ch; // 若是非空串, 则按串长度分配存储区, 否则ch为NULL
    int length; // 串的当前长度
}HString;
```

链式存储

```
#define CHUNKSIZE 80 // 可由用户定义的块的大小
typedef struct Chunk{
    char ch[CHUNKSIZE];
    struct Chunk *next;
}Chunk;
typedef struct{
    Chunk *head, *tail; // 串的头和尾指针
    int length; // 串的当前长度
}
```

3.串的模式匹配算法-暴力版

一旦某个位置失配

模式串整体向右移动一位

从模式串的第一个字符重新开始比较

4.串的模式匹配算法-KMP

思想

利用已匹配部分的结构特征, 直接将模式串向后滑动若干位, 同时主串指针无需回退

next数组

求法

前面子串的最大公共前后缀长度+1

从0开始

next[0]默认为0, next[1]默认为1

next数组值告诉我们应该移动模式串的**第几个**字符到指针下

从1开始

next[0]默认为-1, next[1]默认为0

next数组值告诉我们应该移动模式串**下标为几**的字符到指针下

nextval数组

求法

对比【本位字符】和【本位字符的next数组值对应的字符】

如果二者**相同**, 则nextval值改为next数组值对应的字符的nextval数组值

如果二者**不同**, nextval值和本位的next数组值相同